

Angular 20 CRUD Application (Module-Based) - Full Detailed Guide

STEP 1: Create Angular 20 Module-Based App

```
ng new angular20-crud-app --no-standalone
cd angular20-crud-app
ng serve
```

STEP 2: Install JSON Server

```
npm install json-server --save-dev
```

Create db.json in project root:

```
{
  "products": [
    { "id": 1, "name": "Laptop", "price": 50000 },
    { "id": 2, "name": "Mobile", "price": 20000 }
  ]
}
```

Add script in package.json:

```
"json-server": "json-server --watch db.json --port 3000"
```

Run:

```
npm run json-server
```

API URL:

```
http://localhost:3000/products
```

STEP 3: Configure app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';
import { AppRoutingModuleModule } from './app-routing.module';
import { AppComponent } from './app.component';
```

```
@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    HttpClientModule,
    AppRoutingModuleModule
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

STEP 4: App Routing (Lazy Loading)

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  {
    path: 'products',
    loadChildren: () =>
      import('./features/products/product.module')
        .then(m => m.ProductModule)
  },
  { path: '', redirectTo: 'products', pathMatch: 'full' }
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModuleModule {}
```

STEP 5: Product Module

```
-----
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ReactiveFormsModule } from '@angular/forms';
import { ProductRoutingModule } from '../product-routing.module';
import { ProductListComponent } from '../product-list/product-list.component';
import { ProductFormComponent } from '../product-form/product-form.component';
import { ProductService } from '../product.service';

@NgModule({
  declarations: [
    ProductListComponent,
    ProductFormComponent
  ],
  imports: [
    CommonModule,
    ReactiveFormsModule,
    ProductRoutingModule
  ],
  providers: [ProductService]
})
export class ProductModule {}
```

STEP 6: Product Routing

```
-----
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { ProductListComponent } from '../product-list/product-list.component';
import { ProductFormComponent } from '../product-form/product-form.component';

const routes: Routes = [
  { path: '', component: ProductListComponent },
  { path: 'add', component: ProductFormComponent },
  { path: 'edit/:id', component: ProductFormComponent }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class ProductRoutingModule {}
```

STEP 7: Product Service (CRUD)

```
-----
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class ProductService {

  private apiUrl = 'http://localhost:3000/products';

  constructor(private http: HttpClient) {}

  getAll(): Observable<any> {
    return this.http.get(this.apiUrl);
  }

  getById(id: number): Observable<any> {
    return this.http.get(`${this.apiUrl}/${id}`);
  }

  create(product: any): Observable<any> {
    return this.http.post(this.apiUrl, product);
  }

  update(id: number, product: any): Observable<any> {
    return this.http.put(`${this.apiUrl}/${id}`, product);
  }

  delete(id: number): Observable<any> {
    return this.http.delete(`${this.apiUrl}/${id}`);
  }
}
```

```
}
```

STEP 8: Product List Component

```
import { Component, OnInit } from '@angular/core';
import { ProductService } from '../product.service';
import { Router } from '@angular/router';
```

```
@Component({
  templateUrl: './product-list.component.html'
})
export class ProductListComponent implements OnInit {

  products: any[] = [];

  constructor(private service: ProductService,
               private router: Router) {}

  ngOnInit(): void {
    this.loadData();
  }

  loadData() {
    this.service.getAll().subscribe(res => {
      this.products = res;
    });
  }

  delete(id: number) {
    this.service.delete(id).subscribe(() => {
      this.loadData();
    });
  }
}
```

```
HTML:
<h2>Products</h2>
<button routerLink="add">Add Product</button>
<table border="1">
  <tr>
    <th>Name</th>
    <th>Price</th>
    <th>Actions</th>
  </tr>
  <tr *ngFor="let p of products">
    <td>{{ p.name }}</td>
    <td>{{ p.price }}</td>
    <td>
      <button [routerLink]="['edit', p.id]">Edit</button>
      <button (click)="delete(p.id)">Delete</button>
    </td>
  </tr>
</table>
```

STEP 9: Product Form Component

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { ProductService } from '../product.service';
```

```
@Component({
  templateUrl: './product-form.component.html'
})
export class ProductFormComponent implements OnInit {

  form!: FormGroup;
  id!: number;

  constructor(private fb: FormBuilder,
               private service: ProductService,
               private route: ActivatedRoute,
               private router: Router) {}

  ngOnInit(): void {
```

```

    this.form = this.fb.group({
      name: [''],
      price: ['']
    });

    this.id = this.route.snapshot.params['id'];

    if (this.id) {
      this.service.getById(this.id)
        .subscribe(data => this.form.patchValue(data));
    }
  }

  submit() {
    if (this.id) {
      this.service.update(this.id, this.form.value)
        .subscribe(() => this.router.navigate(['/products']));
    } else {
      this.service.create(this.form.value)
        .subscribe(() => this.router.navigate(['/products']));
    }
  }
}

```

HTML:

```

<h2>Product Form</h2>
<form [formGroup]="form" (ngSubmit)="submit()">
  Name: <input formControlName="name"><br>
  Price: <input formControlName="price"><br>
  <button type="submit">Save</button>
</form>

```

END OF GUIDE